

Generación de servicios en red



Caso práctico

Esta mañana **Juan** ha llegado muy temprano a la empresa. Hay que dar una solución eficiente a la aplicación corporativa de la asociación de periodistas XXL, una aplicación que según cuentan los clientes se ha vuelto muy perezosa, muy lenta, y constantemente se bloquea, o da la impresión de que se bloquea.

La aplicación tiene como cometido proporcionar diferentes servicios en red como impresión, almacenamiento y distribución de archivos así como un sistema de mensajería interna. También gestiona su propio servidor web interno.

Esta aplicación fue diseñada para dar servicio a una Intranet de no más de 5 usuarios y, en la actualidad, la asociación cuenta con más del doble. Además se prevé que en poco tiempo sea necesario conectar la sede principal con una nueva sucursal de reciente creación.

Juan le ha estado dando vueltas, y después de contrastar y consensuar opiniones con **Ada, María y Ana**, parece que ya lo tiene algo más claro: ¿soportarán algunos de los servicios programados en la aplicación comunicaciones simultáneas? Todos apuestan a que no.

Por otra parte, ya puestos a revisar código, la asociación ha encargado nuevas funcionalidades para su aplicación en red, por lo que **Juan** anda muy liado buscando sus apuntes de un Máster que realizó, no hace mucho, sobre generación de servicios en red con Java. Quiere comenzar a implementar los nuevos requisitos.



Q Ministerio de
Educación (Uso educativo nc)



Q  Ministerio de Educación, Formación Profesional y Deportes (Dominio público)

**Materiales formativos de FP Online propiedad del Ministerio de Educación,
Formación Profesional y Deportes.**

 [Aviso Legal](#) 

1.- Introducción

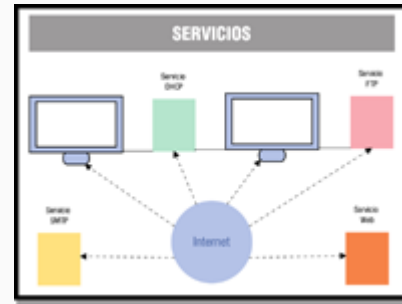
Una red de ordenadores o red informática la podemos definir como un sistema de comunicaciones que conecta ordenadores y otros equipos informáticos entre sí, con la finalidad de compartir información y recursos.

Mediante la compartición de información y recursos en una red, los usuarios de la misma pueden hacer un mejor uso de ellos y, por tanto, mejorar el rendimiento global del sistema u organización.

Las **principales ventajas de utilizar una red de ordenadores** las podemos resumir en las siguientes:

- Reducción en el presupuesto para software y hardware.
- Posibilidad de organizar grupos de trabajo.
- Mejoras en la administración de los equipos y las aplicaciones.
- Mejoras en la integridad de los datos.
- Mayor seguridad para acceder a la información.
- Mayor facilidad de comunicación.

Pues bien, **los servicios en red son responsables directos de estas ventajas.**



Q. Ministerio de Educación (Uso educativo nc)

Un servicio en red es un software o programa que proporciona una determinada funcionalidad o utilidad al sistema. Por lo general, estos programas están basados en un conjunto de protocolos y estándares

Una **clasificación de los servicios en red** atendiendo a su finalidad o propósito puede ser la siguiente:

- **Administración/Configuración.** Esta clase de servicios facilita la administración y gestión de las configuraciones de los distintos equipos de la red, por ejemplo: los servicios DHCP y DNS.
- **Acceso y control remoto.** Los servicios de acceso y control remoto, se encargan de permitir la conexión de usuarios a la red desde lugares remotos, verificando su identidad y controlando su acceso, por ejemplo Telnet y SSH.
- **De Ficheros.** Los servicios de ficheros consisten en ofrecer a la red grandes capacidades de almacenamiento para descongestionar o eliminar los discos de las estaciones de trabajo, permitiendo tanto el almacenamiento como la transferencia de ficheros, por ejemplo FTP.

- **Impresión.** Permite compartir de forma remota impresoras de alta calidad, capacidad y coste entre múltiples usuarios, reduciendo así el gasto.
- **Información.** Los servicios de información pueden servir ficheros en función de sus contenidos, como pueden ser los documentos de hipertexto, por ejemplo HTTP, o bien, pueden servir información para ser procesada por las aplicaciones, como es el caso de los servidores de bases de datos.
- **Comunicación.** Permiten la comunicación entre los usuarios a través de mensajes escritos, por ejemplo email o correo electrónico mediante el protocolo SMTP.

A veces, **un servicio toma como nombre, el nombre del protocolo del nivel de aplicación en el que está basado**. Por ejemplo, hablamos de servicio FTP, por basarse en el protocolo FTP.

En esta unidad, verás ejemplos de cómo programar en Java algunos de estos servicios, pero antes vamos a ver qué son los protocolos del nivel de aplicación.



Para saber más

Consulta el siguiente [enlace](#) si quieres profundizar en los servicios de red.

2.- Protocolos de comunicaciones del nivel de aplicación



Caso práctico



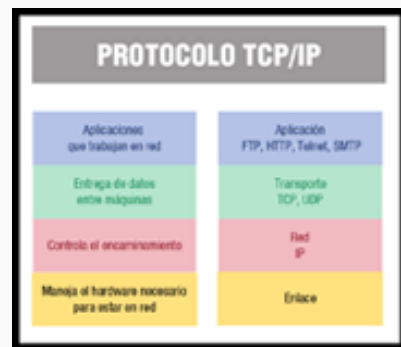
Q Ministerio de Educación (Uso educativo [nc](#))

Ana está entusiasmada con el nuevo reto y no quiere perder detalle. Le dice a **Juan**: "entonces.... ¿seguiremos utilizando **sockets** para los servicios que vamos añadir a la aplicación?"

Juan le responde -Los **sockets** siempre son una opción para programar servicios y clientes, pero para los que están basados en protocolos estándar, podemos encontrar algunas API que facilitan la tarea. Lo

primero que haremos es ver algunos de los protocolos estándar del nivel de aplicación, porque quizá alguno de ellos nos de la respuesta a lo que buscamos.

En la actualidad, el conjunto de protocolos TCP/IP constituyen el modelo más importante sobre el que se basa la comunicación de aplicaciones en red. Esto es debido, no sólo al espectacular desarrollo que ha tenido Internet, en los últimos años (recuerda que TCP/IP es el protocolo que utiliza Internet), sino porque también, TCP/IP ha ido cobrando cada vez más protagonismo en las redes locales y corporativas.



Q Ministerio de Educación (Uso educativo [nc](#))

Dentro de la jerarquía de protocolos TCP/IP **la capa de Aplicación ocupa el nivel superior y es precisamente la que incluye los protocolos de alto nivel relacionados con los servicios en red** que te hemos indicado antes.

La **capa de Aplicación define los protocolos (normas y reglas) que utilizan las aplicaciones para intercambiar datos**. En realidad, hay tantos protocolos de nivel de aplicación como aplicaciones distintas; y además, continuamente se desarrollan nuevas aplicaciones, por lo que el número de protocolos y servicios sigue creciendo.

En la imagen puedes ver un esquema de la familia de protocolos TCP/IP y su organización en capas o niveles.

A continuación, vamos a destacar, por su importancia y gran uso, **algunos de los protocolos estándar del nivel de aplicación**:

- **FTP**. Protocolo para la transferencia de ficheros.
- **Telnet**. Protocolo que permite acceder a máquinas remotas a través de una red. Permite manejar por completo la computadora remota mediante un intérprete de comandos.
- **SMTP**. Protocolo que permite transferir correo electrónico. Recurre al protocolo de oficina postal POP para almacenar mensajes en los servidores, en sus versiones POP2 (dependiente de SMTP para el envío de mensajes) y POP3 (independiente de SMTP).
- **HTTP**. Protocolo de transferencia de hipertexto.
- **SSH**. Protocolo que permite gestionar remotamente a otra computadora de la red de forma segura.
- **NNTP**. Protocolo de Transferencia de Noticias (en inglés Network News Transfer Protocol).
- **IRC**. Chat Basado en Internet (en inglés Internet Relay Chat)
- **DNS**. Protocolo para traducir direcciones de red.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

Ejemplos de protocolos del nivel de aplicación son: FTP, TCP, SMTP, HTTP.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, TCP es un protocolo de nivel de transporte.

2.1.- Comunicación entre aplicaciones

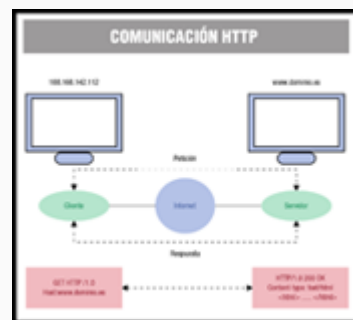
TCP/IP funciona sobre el concepto del modelo cliente/servidor, donde, como recordarás de unidades anteriores:

- **El Cliente** es el programa que ejecuta el usuario y solicita un servicio al servidor. Es quien inicia la comunicación.
- **El Servidor** es el programa que se ejecuta en una máquina (o varias) de red y ofrece un servicio (FTP, web, SMTP, etc.) a uno o múltiples clientes. Este proceso permanece a la espera de peticiones, las acepta y como respuesta, proporciona el servicio solicitado.

La capa de aplicación es el nivel que utilizan los programas para comunicarse a través de una red con otros programas. Los procesos que aparecen en esta capa, son aplicaciones específicas que pasan los datos al nivel de aplicación en el formato que internamente use el programa, y es codificado de acuerdo con un protocolo estándar. Una vez que los datos de la aplicación han sido codificados, en un protocolo estándar del nivel de aplicación, se pasan hacia abajo al siguiente nivel de la pila de protocolos TCP/IP.

En el nivel de transporte, las aplicaciones normalmente hacen uso de TCP y UDP, y son habitualmente asociados a un número de puerto. Por ejemplo, el servicio web mediante HTTP utiliza por defecto el puerto 80, el servicio FTP el puerto 21, etc.

Por ejemplo, la World Wide Web o Web, que usa el protocolo HTTP, sigue fielmente el modelo cliente/servidor:



Q Ministerio de
Educación (Uso educativo no)

- Los clientes son las aplicaciones que permiten consultar las páginas web (navegadores) y los servidores, las aplicaciones que suministran (sirven) páginas web.
- Cuando un usuario introduce una dirección web en el navegador (una URL), por ejemplo `http://www.dominio.es`, éste solicita mediante el protocolo HTTP la página web al servidor web que se ejecuta en la máquina donde está la página.
- El servidor web envía la página por Internet al cliente (navegador) que la solicitó, y éste último la muestra al usuario.

Los dos extremos dialogan siguiendo un protocolo de aplicación, tal y como puedes ver en la imagen que representa el ejemplo anterior.

- El cliente solicita el recurso web `www.dominio.es` mediante `GET HTTP/1.0`
- El servidor le contesta `HTTP/1.0 200 OK` (todo correcto) y le envía la página html solicitada.

Más adelante veremos varios ejemplos de cómo implementar un servicio web básico.



Para saber más

En el siguiente enlace dispones de una relación detallada de números de puerto y servicios que por defecto se le asocian, así como el protocolo de nivel de transporte que utilizan.

[Servicios y puertos asociados](http://es.wikipedia.org/wiki/Anexo:N%C3%BAmeros_de_puerto)  (http://es.wikipedia.org/wiki/Anexo:N%C3%BAmeros_de_puerto).

Los RFC (Request For Comments) documentan los protocolos de aplicación estándar. En el siguiente enlace puedes consultar los RFC.

[Documentación RFC](http://www.rfc-editor.org/)  (<http://www.rfc-editor.org/>).

2.2.- Conexión, transmisión y desconexión

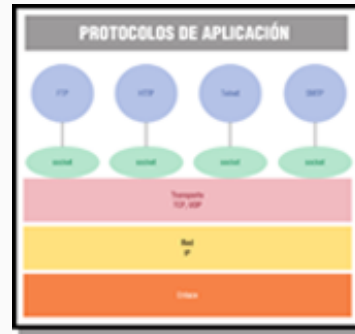
Los protocolos de aplicación se comunican con el nivel de transporte mediante un API, denominada **API Socket**, y que en el caso de Java viene implementada mediante las clases del paquete `java.net` como `Socket` y `ServerSocket`.

Como recordarás, un socket Java es la representación de una conexión para la transmisión de información entre dos ordenadores distintos o entre un ordenador y él mismo. Esta abstracción de medio nivel, permite despreocuparse de lo que está pasando en capas inferiores.

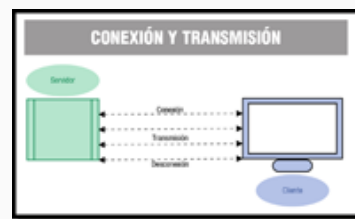
Dentro de una red, un socket es único pues viene caracterizado por cinco parámetros: el protocolo usado (HTTP, FTP, etc.), dos direcciones IP (la del equipo local o donde reside el proceso cliente, y la del equipo remoto o donde reside el proceso servidor) y dos puertos (uno local y otro remoto)

Te recordamos los pasos que se siguen para establecer, mantener y cerrar una conexión TCP/IP:

- Se crean los sockets en el cliente y el servidor
- El servidor establece el puerto por el que proporciona el servicio
- El servidor permanece a la escucha de las peticiones de los clientes.
- Un cliente conecta con el servidor.
- El servidor acepta la conexión.
- Se realiza el intercambio de datos.
- El cliente o el servidor, o ambos, cierran la conexión.



Q Ministerio de Educación (Uso educativo nc)



Q Ministerio de Educación (Uso educativo nc)



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

Los procesos de aplicación se comunican con la capa de transporte mediante un API.

☒ Verdadero ☐ Falso

Verdadero

La afirmación es verdadera, este API es el API socket.

2.3.- DNS y resolución de nombres

Todas las computadoras y dispositivos conectados a una red TCP/IP se identifican mediante una dirección IP, como por ejemplo

117.142.234.125.

```
usuario@usuario-MRC-WX0: ~  
usuario@usuario-MRC-WX0: $ host www.org  
www.org has address 217.70.184.30  
www.org mail is handled by 10 spool.mail.gandi.net.  
www.org mail is handled by 50 fb.mail.gandi.net.  
usuario@usuario-MRC-WX0: $ host www.de  
www.de has address 183.224.182.245  
www.de mail is handled by 10 park-mx.above.com.  
usuario@usuario-MRC-WX0: $
```

Q Ministerio de Educación (Uso educativo nc)

En su forma actual (IPv4), una dirección IP se compone de cuatro bytes sin signo (de 0 a 254) separados por puntos para que resulte más legible, tal y como has visto en el ejemplo anterior. Por supuesto, se trata de valores ideales para los ordenadores, pero para los seres humanos que quieran acordarse de la IP de un nodo en concreto de la red, son todo un problema de memoria.

El sistema DNS o Sistema de Nombres de Dominio es el mecanismo que se inventó, para que los nodos que ofrecen algún tipo de servicio interesante tengan un nombre fácil de recordar, lo que se denomina un nombre de dominio, como por ejemplo www.todoftp.es.

DNS es un sistema de nomenclatura jerárquica para computadoras, servicios o cualquier recurso conectado a Internet o a una red privada.

El objetivo principal de los nombres de dominio y del servicio DNS, es traducir o resolver nombres (por ejemplo www.dominio.es) en las direcciones IP (identificador binario) de cada equipo conectado a la red, con el propósito de poder localizar y direccionar estos equipos dentro de la red.

Sin la ayuda del servicio DNS, los usuarios de una red TCP/IP tendrían que acceder a cada servicio de la misma utilizando la dirección IP del nodo.

Además **el servicio DNS proporciona otras ventajas:**

- Permite que una misma dirección IP sea compartida por varios dominios.
- Permite que un mismo dominio se corresponda con diferentes direcciones IP.
- Permite que cualquier servicio de red pueda cambiar de nodo, y por tanto de IP, sin cambiar de nombre de dominio.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El servicio DNS permite resolver una IP en su único nombre de dominio.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, pues una misma IP puede ser compartida por más de un dominio.



Para saber más

Puedes profundizar en el funcionamiento del servicio DNS en el siguiente enlace:

Información detallada sobre el servicio DNS.  (http://es.wikipedia.org/wiki/Domain_Name_System)

2.4.- El protocolo FTP

El protocolo FTP o Protocolo de Transferencia de Archivos proporciona un mecanismo estándar de transferencia de archivos entre sistemas a través de redes TCP/IP.

Las **principales prestaciones de un servicio basado en el protocolo FTP o servicio FTP** son las siguientes:

- Permite el intercambio de archivos entre máquinas remotas a través de la red.
- Consigue una conexión y una transferencia de datos muy rápidas.

Sin embargo, **el servicio FTP adolece de una importante deficiencia en cuanto a seguridad:**

- La información y contraseñas se transmiten en texto plano. Esto está diseñado así, para conseguir una mayor velocidad de transferencia. Al realizar la transmisión en texto plano, un posible atacante puede capturar este tráfico, acceder al servidor y/o apropiarse de los archivos transferidos.

Este problema de seguridad, se puede solucionar mediante la encriptación de todo el tráfico de información, a través del protocolo no estándar SFTP usando SSH o del protocolo FTPS usando SSL. De encriptación ya hablaremos más adelante, en otra unidad de este módulo.

¿Cómo funciona el servicio FTP? Es un servicio basado en una arquitectura cliente/servidor y, como tal, seguirá las pautas generales de funcionamiento de este modelo, en ese caso:

- El servidor proporciona su servicio a través de dos puertos:
 - El puerto 20, para transferencia de datos.
 - El puerto 21, para transferencia de órdenes de control, como conexión y desconexión.
- El cliente se conecta al servidor haciendo uso de un puerto local mayor de 1024 y tomando como puerto destino del servidor el 21.

Las **principales características del servicio FTP** son las siguientes:

- La **conexión de un usuario remoto** al servidor FTP puede hacerse como: usuario del sistema (debe tener una cuenta de acceso), usuario genérico (utiliza la cuenta anonymous, esto es, usuario anónimo), usuario virtual (no requiere una cuenta local del sistema).
- El **acceso al sistema de archivos** es más o menos limitado, según el tipo de usuario que se conecte y sus privilegios. Por ejemplo, el usuario anónimo solo tendrá acceso al directorio público que establece el administrador por defecto.
- El servicio FTP soporta dos **modos de conexión**: modo activo y modo pasivo.
 - **Modo activo.** Es la forma nativa de funcionamiento (esquema ampliable de la derecha).



Q Gabiwxp (CC BY-SA)

- Se establecen **dos conexiones distintas**: la petición de transferencia por parte del cliente y la atención a dicha petición, iniciada por el servidor. De manera que, si el cliente está protegido con un cortafuegos, deberá configurarlo para que permita esta petición de conexión entrante a través de un puerto que, normalmente, está cerrado por motivos de seguridad.
- **Modo pasivo**. El cliente sigue iniciando la conexión, pero el problema del cortafuegos se traslada al servidor.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

FTP es un servicio que utiliza el puerto 21 para transmitir datos y el 20 para transmitir órdenes de control.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, es justo lo contrario.



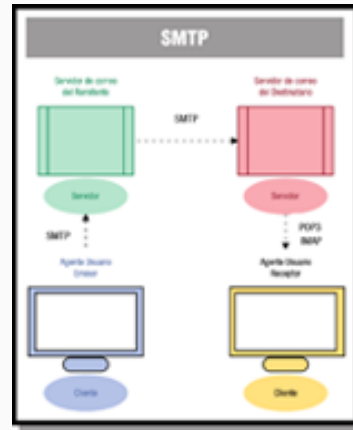
Para saber más

Te recomendamos el siguiente enlace para seguir profundizando en el [Protocolo FTP](#)

2.5.- Los protocolos SMTP y POP3

El correo electrónico es un servicio que permite a los usuarios enviar y recibir mensajes y archivos rápidamente a través de la red.

- Principalmente se usa este nombre para denominar al sistema que provee este servicio en Internet, mediante el protocolo SMTP o Protocolo Simple de Transferencia de Correo, aunque por extensión también puede verse aplicado a sistemas análogos que usen otras tecnologías.
- Por medio de mensajes de correo electrónico se puede enviar, no solamente texto, sino todo tipo de documentos digitales.



Q Ministerio de Educación (Uso educativo nc)

El servicio de correo basado en el protocolo SMTP sigue el modelo cliente/servidor, por lo que el trabajo se distribuye entre el programa Servidor y el Cliente. Te indicamos a continuación, **algunas consideraciones importantes sobre el servicio de correo a través de SMTP**:

- El servidor mantiene las cuentas de los usuarios así como los buzones correspondientes.
- Los clientes de correo gestionan la descarga de mensajes así como su elaboración.
- El servicio SMTP utiliza el puerto 25.
- El protocolo SMTP se encarga del transporte del correo saliente desde la máquina del usuario remitente hasta el servidor que almacene los mensajes de los destinatarios.
 - El usuario remitente redacta su mensaje y lo envía hacia su servidor de correo.
 - Desde allí se reenvía al servidor del destinatario, quién lo descarga del buzón en la máquina local mediante el protocolo POP3, o la consulta vía web, haciendo uso del protocolo IMAP.

¿Cómo funciona SMTP y qué mensajes intercambian cliente y servidor? En el siguiente [enlace](#) dispones de una explicación del funcionamiento del protocolo SMTP en líneas generales (Resumen simple de funcionamiento):



Para saber más

Si quieres profundizar sobre el protocolo POP3, consulta el siguiente enlace:

[Protocolo POP3](http://es.wikipedia.org/wiki/Post_Office_Protocol) (http://es.wikipedia.org/wiki/Post_Office_Protocol)

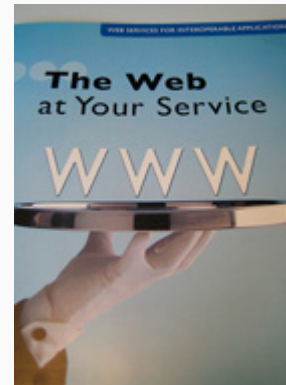
En este otro enlace descubrirás las ventajas de IMAP sobre POP3:

Protocolo IMAP  (http://es.wikipedia.org/wiki/Internet_Message_Access_Protocol)

2.6.- El protocolo HTTP

El protocolo HTTP o Protocolo de Transferencia de Hipertexto es un conjunto de normas que posibilitan la comunicación entre servidor y cliente, permitiendo la transmisión de información entre ambos. La información transferida son las conocidas páginas HTML o páginas web. Se trata del método más común de intercambio de información en la World Wide Web.

HTTP define tanto la sintaxis como la semántica que utilizan clientes y servidores para comunicarse. Algunas **consideraciones importantes sobre HTTP** son las siguientes:



Q Paul Downey (CC BY-SA)

- Es un protocolo que sigue el **esquema petición-respuesta** entre un cliente y un servidor.
- Utiliza por defecto el **puerto 80**
- Al **cliente que efectúa la petición**, por ejemplo un navegador web, se le conoce como agente del usuario (user agent).
- A la **información transmitida se la llama recurso y se la identifica mediante un localizador uniforme de recursos (URL)**.
Por ejemplo: `http://www.iesalandalus.org/organizacion.htm`
- **Los recursos pueden ser** archivos, el resultado de la ejecución de un programa, una consulta a una base de datos, la traducción automática de un documento, etc.

El **funcionamiento esquemático del protocolo HTTP** es el siguiente:

- El usuario especifica en el cliente web la dirección del recurso a localizar con el siguiente formato: `http://dirección[:puerto][ruta]`, por ejemplo: `http://www.iesalandalus.org/organizacion.htm`
- El cliente web descompone la información de la URL diferenciando el protocolo de acceso, la IP o nombre de dominio del servidor, el puerto y otros parámetros si los hubiera.
- El cliente web establece una conexión al servidor y solicita el recurso web mediante un mensaje al servidor, encabezado por un método, por ejemplo `GET /organizacion.htm HTTP/1.1` y otras líneas.
- El servidor contesta con un mensaje encabezado con la línea `HTTP/1.1 200 OK`, si existe la página y la envía, o bien envía un código de error.
- El cliente web interpreta el código HTML recibido.
- Se cierra la conexión.

En el siguiente enlace dispones de un [ejemplo de diálogo entre cliente y servidor usando el protocolo HTTP](#). Más adelante, estudiaremos algunos detalles más de este protocolo para poder programar un servicio web básico.

HTTP es un protocolo sin estado, lo que significa que no recuerda nada relativo a

conexiones anteriores a la actual. Algunas aplicaciones necesitan que se guarde el estado y para ello hacen uso de lo que se conoce como cookie.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

HTTP es un protocolo al igual que URL.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, http es un protocolo, pero URL no es un protocolo, es la dirección de un recurso en la web.



Para saber más

¿No sabes que son y para que sirven las cookies? Consulta este enlace para aprender sobre ellas.

Información sobre cookies  (http://es.wikipedia.org/wiki/Cookie_%28inform%C3%A1tica%29#Prop.C3.B3sito)

Si quieres profundizar en el funcionamiento del protocolo HTTP te recomendamos que visites este enlace:

Información sobre protocolo HTTP  (https://es.wikipedia.org/wiki/Protocolo_de_transferencia_de_hipertexto)

3.- Bibliotecas de clases y componentes Java



Caso práctico

Ana le dice a **Juan**, -ahora tenemos que ver qué clases nos proporciona Java para implementar servicios y clientes en red ¿Habría alguna otra además de `java.net` ?.

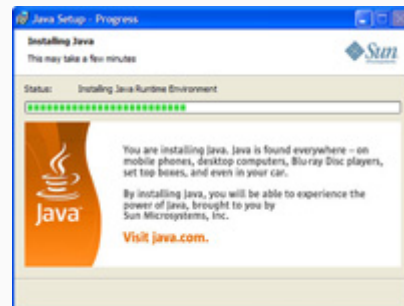
Juan le contesta, -la biblioteca `java.net` es muy extensa y es la que proporciona prácticamente todas las funcionalidades de programación en red. Vamos a estudiarla a fondo y ver qué clases nos pueden servir para nuestra aplicación.

Ana dice, -me parece perfecto. ¿Empezamos?



Q Ministerio de Educación (Uso educativo nc)

Java se ha construido con extensas capacidades de interconexión TCP/IP y soporta diferentes niveles de conectividad en red, facilitando la creación de aplicaciones cliente/servidor y generación de servicios en red. Así por ejemplo: permite abrir una URL como por ejemplo



Q hillary h (CC BY-SA)

`http://www.dominio.es/public/archivo.pdf`, permite realizar una invocación de métodos remotos, RMI, o permite trabajar con sockets.

El paquete principal que proporciona el API de Java para programar aplicaciones con comunicaciones en red es:

- `java.net`. Este paquete soporta clases para generar diferentes servicios de red, servidores y clientes.

Otros paquetes Java para comunicaciones y servicios en red son::

- `java.rmi`. Paquete que permite implementar una interface de control remoto (RMI).
- `javax.mail`. Permite implementar sistemas de correo electrónico.

Para ciertos servicios estándar, Java no proporciona objetos predefinidos y por tanto una solución fácil para generarlos es recurrir a bibliotecas externas, como por ejemplo, el API proporcionado por Apache Commons Net (las bibliotecas `org.apache.commons.net`). Esta API permite implementar aplicaciones cliente para los protocolos estándar más populares, como: Telnet, FTP, o FTP sobre HTTP entre otros.



En el siguiente enlace puedes consultar todas las funcionalidades que ofrece el API Apache Commons Net.

Bibliotecas de Apache Commons Net  (<http://commons.apache.org/net/>).

3.1.- Objetos predefinidos

El paquete `java.net`, proporciona una API que se puede dividir en dos niveles:

- Una API de bajo nivel, que permite representar los siguientes objetos:

- **Direcciones.** Son los identificadores de red, esto es, las direcciones IP.
 - Clase `InetAddress`. Implementa una dirección IP.
- **Sockets.** Son los mecanismos básicos de comunicación bidireccional de datos.

- Clase `Socket`. Implementa un extremo de una conexión bidireccional.
- Clase `ServerSocket`. Implementa un `socket` que los servidores pueden utilizar para escuchar y aceptar peticiones de clientes.
- Clase `DatagramSocket`. Implementa un `socket` para el envío y recepción de datagramas.
- Clase `MulticastSocket`. Representa un `socket` datagrama, útil para enviar paquetes multidifusión.
- **Interfaces.** Describen las interfaces de red.
- Clase `NetworkInterface`. Representa una interfaz de red compuesta por un nombre y una lista de direcciones IP asignadas a esta interfaz.

- Una API de alto nivel, que se ocupa de representar los siguientes objetos, mediante las clases que te indicamos:

- **URI.** Representan los identificadores de recursos universales.
 - Clase `URI`.
- **URL.** Representan localizadores de recursos universales.
 - Clase `URL`. Representa una dirección URL.
- **Conexiones.** Representa las conexiones con el recurso apuntado por URL.
 - Clase `URLConnection`. Es la superclase de todas las clases que representan un enlace de comunicaciones entre la aplicación y una URL.
 - Clase `HttpURLConnection`. Representa una `URLConnection` con soporte para HTTP y con ciertas características especiales.



Q Ministerio de Educación (Uso educativo nc)

Mediante las clases de alto nivel se consigue una mayor abstracción, de manera que, esto facilita bastante la creación de programas que acceden a los recursos de la red. A continuación veremos algunos ejemplos.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

La clase `URLConnection` representa una enlace entre un recurso URL y la aplicación.

☐ Verdadero ☐ Falso

Verdadero

La afirmación es verdadera. Es la superclase de todas las clases que representan un enlace de comunicaciones netre la aplicación y la URL.



Para saber más

En el siguiente enlace puedes consultar el paquete `java.net` en toda su extensión:

 Paquete `java.net`  (<https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/net/package-summary.html>)

3.2.- Métodos y ejemplos de uso de InetAddress

La clase `InetAddress` proporciona objetos que puedes utilizar para manipular tanto direcciones IP como nombres de dominio. También proporciona métodos para resolver los nombres de host a sus direcciones IP y viceversa.



Q Jaques Wagner (
CC BY-SA)

Una instancia de `InetAddress` consta de una dirección IP y en algunos casos también del nombre de host asociado. Esto último depende de si se ha creado con el nombre de host o bien ya se ha aplicado la resolución de nombres.

Esta clase no tiene constructores. Sin embargo, `InetAddress` **dispone de métodos estáticos que devuelven objetos `InetAddress`**. Te indicamos cuáles son esos métodos:

- `getLocalHost()`. Devuelve un objeto de tipo `InetAddress` con los datos de direccionamiento de mi equipo en la red local (no del conocido `localhost`).
- `getByName(String host)`. Devuelve un objeto de tipo `InetAddress` con los datos de direccionamiento del host que le pasamos como parámetro. Donde el parámetro `host` tiene que ser un nombre o IP válido, ya sea de Internet (como `iesalandalus.org` o `195.78.228.161`), o de tu red de área local (como `documentos.servidor` o `192.168.0.5`). Incluso puedes poner `localhost`, o cualquier otra IP o nombre NetBIOS de tu red local.
- `getAllByName(String host)`. Devuelve un `array` de objetos de tipo `InetAddress` con los datos de direccionamiento del host pasado como parámetro. Recuerda que en Internet es frecuente que un mismo dominio tenga a su disposición más de una IP.

Todos estos métodos pueden generar una excepción `UnknownHostException` si no pueden resolver el nombre pasado como parámetro.

Algunos **métodos interesantes de un objeto `InetAddress`** para resolver nombres son los siguientes:

- `getHostAddress()`. Devuelve en una cadena de texto la correspondiente IP.
- `getAddress()`. Devuelve un `array` formado por los grupos de bytes de la IP correspondiente.

Otros métodos interesantes de esta clase son:

- `getHostName()`. Devuelve en una cadena de texto el nombre del host.
- `isReachable(int tiempo)`. Devuelve TRUE o FALSE dependiendo de si la dirección es alcanzable en el tiempo indicado en el parámetro.

En el siguiente enlace tienes un ejemplo de uso de `InetAddress`. Con este ejemplo tratamos de ilustrar el uso de los métodos anteriores para resolver algunos nombres a su dirección o direcciones IP, tanto en la red local como en Internet. Por ello, para probarlo, debes tener conexión a Internet, en otro caso sólo se ejecutará la parte relativa a la red local, y se lanzará la correspondiente excepción al intentar resolver nombres de Internet.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `getLocalHost()` devuelve la IP del equipo de nombre `localhost`.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, devuelve un objeto `InetAddress` asociado a mi equipo en la red local.



Citas para pensar

"Es absolutamente imposible demostrarlo todo."

Aristóteles

3.3.- Programación con URL

La programación de **URL** se produce a un nivel más alto que la programación de **sockets** y ésto, puede facilitar la creación de aplicaciones que acceden a recursos de la red.

¿Tienes claro lo que es una URL y las partes en las que se puede descomponer? Vamos a verlo.

Una URL, Localizador Uniforme de Recursos, representa una dirección a un recurso de la World Wide Web. Un **recurso** puede ser algo tan simple como un archivo o un directorio, o puede ser una referencia a un objeto más complicado, como una consulta a una base de datos, el resultado de la ejecución de un programa, etc.

Consideraciones sobre una URL:

- Puede referirse a sitios web, ficheros, sitios ftp, direcciones de correo electrónico, etc.
- La **estructura de una URL** se puede dividir en varias partes, tal y como puede ver en la imagen.
- **Protocolo**. El protocolo que se usa para comunicar.
- **Nombrehost**. Nombre del host que proporciona el servicio o servidor.
- **Puerto**. El puerto de red en el servidor para conectarse. Si no se especifica, se utiliza el puerto por defecto para el protocolo.
- **Ruta**. Es la ruta o path al recurso en el servidor.
- **Referencia**. Es un fragmento que indica una parte específica dentro del recurso especificado.



Q Ministerio de Educación. (Uso educativo nc)

Por ejemplo, algunas posibles direcciones URL serían las siguientes:

- <http://www.iesalandalus.org/organizacion.htm>, el protocolo utilizado es el http, el nombre del host www.iesalandalus.org y la ruta es [organización.htm](http://www.iesalandalus.org/organizacion.htm), que en este caso es una página html. Al no indicar puerto, se toma el puerto por defecto para HTTP que es el 80.
- <http://www.iesalandalus.org:85/organizacion.htm>, en este caso se está indicando como puerto el 85.
- <http://www.dominio.es/public/pag.html#apartado1>, en este caso se especifica como ruta [/public/ pag.html#apartado](http://www.dominio.es/public/pag.html#apartado1), por lo que una vez recuperado el archivo [pag.htm](http://www.dominio.es/public/pag.html#apartado1) se está indicando mediante la referencia [#apartado1](http://www.dominio.es/public/pag.html#apartado1) que interesa el apartado1 dentro de esa página.



Debes conocer

En una URL se pueden especificar otros protocolos además de HTTP. Consulta el siguiente enlace para ver esos protocolos y más ejemplos de

posibles URL.

Partes de una URL y posibles protocolos (se abre ventana nueva)  (http://es.wikipedia.org/wiki/Localizador_uniforme_de_recursos#Esquema_URL).

La clase `URL` de Java permite representar una URL.

Utilizando la clase URL, se puede establecer una conexión con cualquier recurso que se encuentre en Internet, o en nuestra Intranet. Una vez establecida la conexión, invocando los métodos apropiados sobre ese objeto URL se puede obtener el contenido del recurso en el cliente. Esto es una idea potentísima, pues permite, en teoría, descargar el contenido de cualquier recurso en el cliente, incluso aunque se requiriera un protocolo que no existía cuando se escribió el programa.

Pero la clase URL también proporciona una forma alternativa de conectar un ordenador con otro y compartir datos, basándose en `streams`. Esto último, es algo que también se puede hacer con la programación de `sockets`, tal y como has visto en las unidades anteriores.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

En una URL el protocolo que se debe especificar es HTTP.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, pues se admiten otros protocolos como por ejemplo ftp.

3.4.- Crear y analizar objetos URL

¿Cómo podemos crear objetos URL? La clase `URL` dispone de diversos constructores para crear objetos tipo URL que se diferencian en la forma de pasarle la dirección URL. Por ejemplo, se puede crear un objeto `URL` :



Q Ben Sheldon (CC BY-SA)

- A partir de todos o algunos de los elementos que forman parte de una URL, como por ejemplo:
`URL url=new URL("http", "www.iesalandalus.org",80,"index.htm")`.
Crea un objeto `URL` a partir de los componentes indicados (protocolo, host, puerto, fichero), esto es, crea la URL: `http://www.iesalandalus.org:80/index.htm`.
- A partir de la cadena especificada, dejando que el sistema utilice todos los valores por defecto que tiene definidos, como por ejemplo:
`URL url=new URL("http://www.iesalandalus.org")` que crearía la URL : `http://www.iesalandalus.org`.
- A partir de una ruta relativa pasada como primer parámetro.
- A partir de las especificaciones indicadas y un manejador del protocolo que conoce cómo comunicarse con el servidor.

Cada uno de los constructores de URL puede lanzar una `MalformedURLException` si los argumentos del constructor son nulos o el protocolo es desconocido. Lo normal, es capturar y manejar esta excepción mediante un bloque `try- catch`.

En el siguiente [ENLACE](#) dispones de ejemplos que ilustran diferentes formas de crear un objeto URL.

Las URLs son objetos de una sola escritura. Lo que significa, que una vez que has creado un objeto URL no se puede cambiar ninguno de sus atributos (protocolo, nombre del host, nombre del fichero ni número de puerto).

Se puede analizar y descomponer una URL utilizando los siguientes métodos:

- `getProtocol()` . Obtiene el protocolo de la URL.
- `getHost()` . Obtiene el host de la URL.
- `getPort()` . Obtiene el puerto de la URL, si no se ha especificado obtiene -1.
- `getDefaultPort()` . Obtiene el puerto por defecto asociado a la URL, si no lo tiene obtiene -1.
- `getFile()` . Obtiene el fichero de la URL o una cadena vacía si no existe.

- `getRef()` . Obtiene la referencia de la URL o `null` si no la tiene.

En el siguiente [ENLACE](#) puedes ver un ejemplo de cómo analizar o descomponer una URL.



Autoevaluación

Señala las opciones correctas. Métodos que permiten analizar una URL son:

- ☐ El método `getURL()` que obtiene cada parte de la URL.
- ☐ El método `getHost()` que obtiene el host.
- ☐ El método `URL("direccion")` que la analiza de forma completa.
- ☐ El método `getProtocol()` que obtiene el protocolo.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-45_58)
2. Correcto (#answer-45_103)
3. Incorrecto (#answer-45_105)
4. Correcto (#answer-45_107)

3.5.- Leer y escribir a través de URLConnection

Un objeto `URLConnection` se puede utilizar para leer desde y escribir hacia el recurso al que hace referencia el objeto `URL`.



Q dimah2381 (CC BY-SA)

De entre los muchos **métodos** que nos permiten **trabajar con conexiones URL** vamos a centrarnos en primer lugar en los siguientes:

- `URL.openConnection()`. Devuelve un objeto `URLConnection` que representa una nueva conexión con el recurso remoto al que se refiere la URL.
- `URL.openStream()`. Abre una conexión a esta dirección URL y devuelve un `InputStream` para la lectura de esa conexión. Es una abreviatura de: `openConnection().getInputStream()`.

Te mostramos a continuación dos ejemplos muy sencillos que leen una URL, basándose tan solo en estos dos métodos. Los pasos a seguir para leer la URL son:

- Crear el objeto `URL` mediante `URL url = URI.create("...").toURL();`
- Obtener una conexión con el recurso especificado mediante `URL.openConnection()`.
- Abrir conexión con esa URL mediante `URL.openStream()`.
- Manejar los flujos necesarios para realizar la lectura

Por ejemplo, podemos utilizar objetos `URL` para leer un archivo de lectura y almacenarlo en un fichero local.

También podemos utilizar objetos `URL` para leer una URL y analizar su código fuente, tal y como hacen los buscadores, optimizadores de código o validadores de código. En el siguiente [ENLACE](#)  dispones de un ejemplo de este uso, desde donde puedes descargar el código y probarlo en tu equipo.

En el siguiente [ENLACE](#)  dispones de más ejemplos para leer y escribir mediante un objeto `URLConnection`.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `URL.openStream()` devuelve un objeto `URLConnection`.

☐ Verdadero ☐ Falso

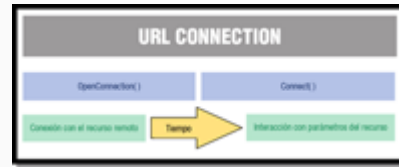
Falso

La afirmación es falsa, devuelve un objeto `InputStream` para la lectura de la URL.

3.6.- Trabajar con el contenido de una URL

El método que permite **obtener** el **contenido de un objeto URL** es:

- `URL.getContent()`. Devuelve el contenido de esa URL. Este método determina en primer lugar el tipo de contenido del objeto llamando al método `getContentType()`. Si esa es la primera vez que la aplicación ha visto ese tipo de contenido específico, se crea un manejador para dicho tipo de contenido.



Q Ministerio de Educación. (Uso educativo nc)

Mediante este método y otros proporcionados por la clase `URLConnection`, veremos al final de este apartado, un ejemplo de cómo examinar el contenido del objeto URL y realizar la tarea que interese.

Crear una conexión URL realmente implica los siguientes pasos:

- Crear un objeto `URLConnection`, éste se crea cuando se llama al método `openConnection()` de un objeto `URL`.
- Establecer los parámetros y propiedades de la petición.
- Establecer la conexión utilizando el método `connect()`.
- Se puede obtener información sobre la cabecera o/y obtener el recurso remoto.

A partir de un objeto `URLConnection`, se puede obtener información muy útil sobre la conexión que se haya establecido y existen muchos métodos en la clase `URLConnection` que se pueden utilizar para examinar y manipular el objeto que se crea de este tipo.

Vamos a ver algunos métodos que serán de utilidad en nuestro ejemplo:

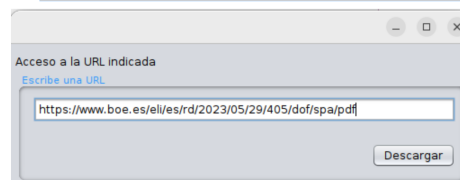
- `connect()`. Establece una conexión entre la aplicación (el cliente) y el recurso (el servidor), permite interactuar con el recurso y consultar los tipos de cabeceras y contenidos para determinar el tipo de recurso de que se trata.
- `getContentType()`. Devuelve el valor del campo de cabecera `content-type` o `null` si no está definido.
- `getContentLength()`. Devuelve el valor del campo de cabecera `content-length` o -1 si no está definido.
- `getLastModified()`. Devuelve la fecha de la última modificación del recurso.

A continuación dispones de una presentación que muestra la creación de una aplicación para acceder a recursos de Internet a través de objetos `URL`, esto es, un cliente web.

Acceso a Recursos

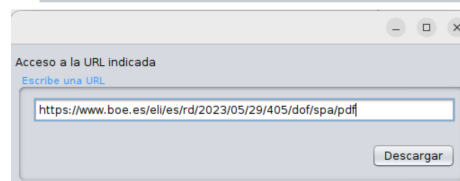
URL y URLConnection

Objetivo del ejemplo (I)



- Se muestra al usuario una ventana con un **recuadro de texto donde puede escribir una URL** (por defecto se trata de un destino que apunta a un documento PDF)
- **Al hacer clic en el botón** se produce la siguiente secuencia de pasos: (ver siguiente diapositiva...)

Objetivo del ejemplo (II)



- Se realiza la conexión al servidor de destino y, en caso de que sea alcanzable, se analiza la respuesta recibida por éste. Para ello es fundamental inspeccionar el valor de la propiedad **'Content-Type'**:
 - si es un documento **pdf** el usuario es preguntado acerca de la ubicación y nombre de cara a proceder a la descarga y almacenado.
 - si fuese un texto o documento **html** lo procesa e imprime en la salida estándar
 - si no se trata de ninguno de los casos anteriores no lo resuelve.

Definición y desarrollo de la interfaz

- Empleando un **JFrame** procedemos agregando algunos controles:
- Dos etiquetas (**JLabel**)
- Un marco (**JPanel**)
- Una caja de texto (**TextField**) y un botón (**Button**) dentro del marco definido anteriormente



- La caja de texto se utiliza para introducir la URL
- El botón **descargar** permite obtener la URL indicada

Lógica asociada a la ventana

- Definir en el constructor la URL por defecto. Consiste simplemente en indicar el valor que aparecerá en el recuadro de texto:
<https://www.boe.es/eli/es/or/2010/07/13/edu2000/dof/spa/pdf>

```
/**
 * *****
 * Constructor de la clase "Ventana". Es invocado por el método 'main' ubicado
 * al final del archivo cuando se inicia la ejecución
 */
public Ventana() {
    // inicialización
    initComponents();
    // especifica una URL de cara a probar el software:
    textField.setText("https://www.boe.es/eli/es/or/2023/05/29/095/dof/spa/pdf");
}
```

- El código asociado al botón **Descargar** inicia la ejecución de un hilo de la clase **HiloBoton** el cual se realiza la tarea de conectar con el recurso especificado en la URL.

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // hilo que realiza la descarga del documento apuntado por una url
    HiloBoton hiloBoton = new HiloBoton(textField.getText());
    hiloBoton.start();
} //GEN-LAST:event_jButton1ActionPerformed
```

Código de la clase HiloBoton (I)

- Se inicia la lógica asociada a la ejecución del hilo para lo cual se definen las variables auxiliares y se realiza la conexión al servidor

```
/**
 * Lógica computacional asociada a la ejecución del hilo
 */
@Override
public void run() {
    // control de bytes descargados del documento
    int leído;
    // tamaño total, en bytes, del documento
    long contentLength;
    // buffer de tipo carácter para almacenamiento temporal
    char[] bufChar;
    // buffer de tipo byte para almacenamiento temporal
    byte[] bufByte;
    // almacenar, en el caso de documento HTML o PDF, el nombre que el
    // servidor devuelve en la cabecera de respuesta, más concretamente
    // el campo el valor "filename" en el campo "Content-Disposition" si
    // es que éste no fuera nulo o vacío
    String nombreDocumento = "";
    try {
        // creación de objeto de tipo URL
        URL url = URL.create(cadenteURL).toURL();
        // creación de objeto que permite la conexión al recurso indicado
        URLConnection conexion = url.openConnection(); // en la URL
    }
```

Código de la clase HiloBoton (II)

[illegible]

- Tras lo anterior se procede a establecer las propiedades:
 - 'Accept-Encoding' con valor 'identity' (desactivar la compresión para conservar los atributos contenidos en la cabecera)
 - Tipo: 'Content-Type'
 - Tamaño: 'Content-Length'
 - Nombre:
 - 'Content-Disposition'
 - 'filename'
 - Fecha de modificación ('getLastModified()')

Código de la clase HiloBoton (III)

[illegible]

- Si el contenido es del tipo `'application/pdf'` se abre un flujo de lectura al recurso que permite guardarlo localmente

Código de la clase HiloBoton (IV)

```
// en el caso de que sea un documento HTML o
else if (contentType.startsWith("text/html")) // texto plano ("text/html")
// creación de flujo para conectar el cuerpo
// del texto plano o el documento HTML
InputStreamReader = conexion.getInputStream();

// definición de buffer de lectura
BufferedReader bufferedReader
    = new BufferedReader(new InputStreamReader(inputStream));

// caso no sobrecarga el recurso reservado para esta finalidad
bufChar = new char[512];

// informamos en la línea estándar que estamos en proceso...
System.out.println("Escribiendo el texto plano o documento HTML."
    + " en la salida estándar");

// mientras haya caracteres pendiente de ser procesados
// (se recuerda que la condición de finalización es recibir el
// valor -1, el cual es verificado cuando sea mayor de 0)
while ((leido = bufferedReader.read(bufChar)) > 0) {
    // se imprime la salida estándar tanto en los casos según son
    System.out.println(new String(bufChar, 0, leido)); // recibidos
}

// informamos del final de la operación
System.out.println("Descarga de texto plano o documento HTML "
    + "correctamente realizado.");
} else { // en otro caso:
```

- Si el contenido es del tipo `'text/html'` abre un flujo de lectura al recurso y, a la vez que lo consume, lo muestra por la salida estándar

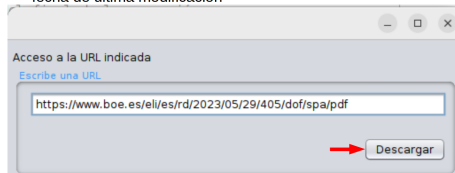
Código de la clase HiloBoton (V)

- En otro caso no se realiza ninguna acción

```
    } else { // en otro caso:
        // Informamos que no se trata de un documento PDF ni de texto plano
        // o documento HTML así pues no hacemos nada
        System.out.println("No se trata de documento PDF, HTML o texto "
            + "plano así pues no realizamos ninguna acción");
    }
} catch (MalformedURLException e) {
    System.err.println("URL malformada: " + e.getMessage());
} catch (IOException e) {
    System.err.println("Error de lectura/escritura: " + e.getMessage());
} catch (Exception e) {
    System.err.println("Error: " + e.getMessage());
} finally {
    System.exit(0); // se finaliza la ejecución del hilo
}
```

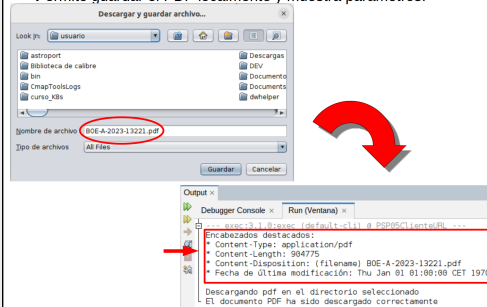
Al ejecutar el programa

- Aparece la ventana con la URL indicada por defecto. Si hacemos clic pulsamos sobre el botón 'Descargar' permite descargar y guardar el documento PDF y, por consola, muestra los parámetros:
- tipo contenido
- tamaño
- nombre de archivo
- fecha de última modificación



En caso de que la URL sea documento PDF

- Permite guardar el PDF localmente y muestra parámetros.



Si es la URL de un HTML

- Si ponemos la URL `'https://www.boe.es'` se descarga el archivo principal que se aloja en dicho servidor



```
Output x
Debugger Console x Run (Ventana) x
Encabezados destacados:
* Content-Type: text/html; charset=UTF-8
* Content-Length: 22646
* Fecha de última modificación: Thu Jan 01 01:00:00 CET 1970
Escribiendo el texto plano o documento HTML en la salida estándar:
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<meta name="format-detection" content="telephone=no" />
<meta name="description" content="Agencia Estatal Boletín Oficial del Estado" />
<title>BOE.es - Agencia Estatal Boletín Oficial del Estado</title>
<link rel="shortcut icon" href="/favicon.ico" />
<link rel="icon" href="/favicon.ico" type="image/x-icon" />
<link rel="apple-touch-icon" href="/apple-touch-icon.png" />
<base target="_top" />
```

Desde el siguiente enlace [ENLACE](#) (12.8 KB) te puedes descargar el proyecto completo:



Para saber más

Se puede utilizar la clase `URL` para escribir manejadores de protocolos o cosas parecidas. Para ello, será necesario conocer en profundidad la clase `URLConnection`. Consulta este enlace para ello.

 [Clase URLConnection](https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/net/URLConnection.html) (https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/net/URLConnection.html)

Desde este enlace puedes consultar todos los parámetros de las cabeceras del protocolo HTTP.

[Campos de la cabecera del protocolo HTTP](#) 

4.- Programación de aplicaciones cliente



Caso práctico

María ha ido hoy a ver a **Juan** y **Ana**, pues **Ada** le ha comentado que quizá le venga bien ver la marcha del proyecto. **María**, que lleva el mantenimiento de varias webs a la empresa, está interesada en conocer más de cerca los entresijos de determinadas aplicaciones de Internet. **Juan** y **Ana** la saludan y **Juan** le dice: -No has podido elegir mejor día para trabajar con nosotros,

María. Justo hoy veremos algunas API que facilitan mucho la programación de ciertos clientes de Internet al trabajar a un nivel alto, por encima de los `sockets`.

María sonríe y dice: -Perfecto, para sorpresa vuestra os diré que conozco un API para programar clientes de correo electrónico. -**Ana** dice- ¿Te refieres a `javax.mail`?



Q Ministerio de Educación. (Uso educativo-nc)



Q Bill Abbott (CC BY-SA)

La programación de aplicaciones cliente y aplicaciones servidor que vamos a realizar, está basada en el uso de los protocolos estándar de la capa de aplicación, por tanto, cuando programemos una aplicación servidor basada en el protocolo HTTP, por ejemplo, diremos también que se está programando un servicio o

servidor HTTP, y si lo que programamos es el cliente, hablaremos de cliente HTTP. Nos vamos a centrar en la programación de aplicaciones cliente para los protocolos HTTP, FTP, SMTP y TELNET. Su programación se realizará mediante bibliotecas especiales que proporcionan ciertos objetos que facilitan la tarea de programación, y en algunos casos veremos también cómo programar esas mismas aplicaciones cliente mediante `sockets`. Es imprescindible un mínimo conocimiento de cómo funciona el protocolo sobre el que vamos a construir un cliente, para tener claro el intercambio de mensajes que realiza con el servidor. Por tanto, no repares en volver a los primeros apartados o consultar los enlaces para saber más, donde se explica algo sobre el funcionamiento de estos protocolos. Esto es más necesario, si la programación se realiza a niveles relativamente bajos, como por ejemplo cuando utilizamos `sockets`.



Citas para pensar

"Las ciencias tienen las raíces amargas, pero muy dulces los frutos."

Aristóteles

4.1.- Programación de un cliente HTTP

Como ya te hemos comentado anteriormente, HTTP se basa en sencillas operaciones de solicitud/respuesta.

- Un cliente establece una conexión con un servidor y envía un mensaje con los datos de la solicitud.
- El servidor responde con un mensaje similar, que contiene el estado de la operación y su posible resultado.



Q JavierRiGa (CC BY-SA)

¿Es un cliente HTTP el ejemplo que hemos visto sobre el acceso a recursos de la red mediante URL, utilizando las clases `URL` y `URLConnection`? Efectivamente, se trataba de un cliente web básico, en particular un cliente http muy básico, que no realiza por ejemplo la traducción de una página html tal y como lo hace un navegador.

Al programar aplicaciones con las clases `URL` y `URLConnection`, lo hacemos en un nivel alto, de manera que queda oculta toda la operatoria que era tan explícita al programar un cliente con `sockets`.

En el siguiente [ENLACE](#) puedes consultar otro ejemplo de cliente HTTP realizado con `sockets` y cuyo comportamiento es similar a los ejemplos vistos anteriormente mediante las clases `URL` s.



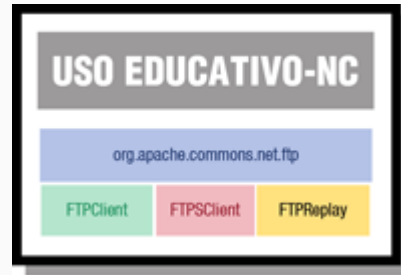
Para saber más

¿Sabías que puedes ver las cabeceras o encabezados que envía el servidor web a tu navegador y viceversa mediante una extensión del navegador Firefox? Mira el siguiente enlace y lo podrás comprobar.

 [Ver las cabeceras que envían navegador y servidor web](http://www.desarrolloweb.com/articulos/cabeceras-http-livehttpheaders.html) (<http://www.desarrolloweb.com/articulos/cabeceras-http-livehttpheaders.html>)

4.2.- Bibliotecas para programar un cliente FTP

Java no dispone de bibliotecas específicas para programar clientes y servidores de FTP. Pero afortunadamente, la organización de software libre The Apache software Foundation proporciona una API para implementar clientes FTP en tus aplicaciones.



Q Ministerio de Educación. (Uso educativo nc)

El **paquete de la API de Apache para trabajar con FTP** es

org.apache.commons.net.ftp. Este paquete proporciona, entre otras, las siguientes **clases**:

- **Clase FTP**. Proporciona las funcionalidades básicas para implementar un cliente FTP. Esta clase hereda de **SocketClient**.
- **Clase FTPReply**. Permite almacenar los valores devueltos por el servidor como código de respuesta a las peticiones del cliente.
- **Clase FTPClient**. Encapsula toda la funcionalidad que necesitamos para almacenar y recuperar archivos de un servidor FTP, encargándose de todos los detalles de bajo nivel para su interacción con el servidor. Esta clase hereda de **SocketClient**.
- **Clase FTPClientConfig**. Proporciona una forma alternativa de configurar objetos **FTPClient**.
- **Clase FTPSCient**. Proporciona FTP seguro, sobre SSL. Esta clase hereda de **FTPClient**.



Para saber más

En el siguiente vídeo se contiene una presentación en la cual se explican los pasos a seguir para descargar una librería de Apache (Apache Commons Net) e integrarla en un proyecto en el IDE NetBeans:

<https://www.youtube.com/embed/PsnVU5l8VJo> (<https://www.youtube.com/embed/PsnVU5l8VJo>)

Q Ministerio de Educación. (Licencia estándar de YouTube)

 Descripción textual del vídeo 



Citas para pensar

"Lo que sabemos es una gota de agua; lo que ignoramos es el océano "

Isaac Newton



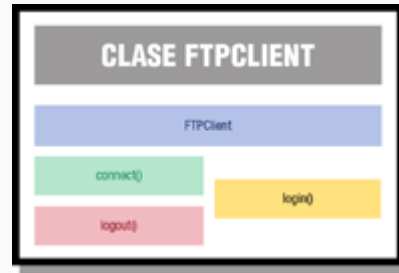
Para saber más

En el siguiente  [ENLACE](#)  dispones de toda la información sobre el API de Apache para el servicio FTP.

4.3.- Programación de un cliente FTP

Una forma sencilla de **crear un cliente FTP** es **mediante la clase `FTPClient`**.

Una vez creado el cliente, como en cualquier objeto `SocketClient` habrá que seguir el siguiente **esquema básico de trabajo**:



Q Ministerio de Educación. (Uso educativo no)

- **Realizar la conexión del cliente con el servidor.** El método `connect(InetAddress host)` realiza la conexión del cliente con el servidor de nombre `host`, abriendo un objeto `Socket` conectado al host remoto en el puerto por defecto.
- **Comprobar la conexión.** El método `getReplyCode()` devuelve un código de respuesta del servidor indicativo de el éxito o fracaso de la conexión.
- **Validar usuario.** El método `login(String usuario, String password)` permite esa validación.
- **Realizar operaciones contra el servidor.** Por ejemplo:
 - Listar todos los ficheros disponibles en una determinada carpeta remota mediante el método `listNames()`.
 - Recuperar el contenido de un fichero remoto mediante `retrieveFile(String rutaRemota, OutputStream ficheroLocal)` y transferirlo al equipo local para escribirlo en el `ficheroLocal` especificado.
- **Desconexión del servidor.** El método `disconnect()` o `logout()` realiza la desconexión del servidor.

Ten en cuenta, que durante todo este proceso puede generarse tanto una `SocketException` si se superó el tiempo de espera para conectar con el servidor, o una `IOException` si no se tiene acceso al fichero especificado.

En el siguiente [ENLACE](#), tienes un ejemplo de programación de un cliente FTP, que se conecta a un servidor remoto y se descarga un fichero. Ten en cuenta que para poder ejecutarlo, puede que tengas que desactivar cualquier [cortafuegos](#) o firewall que tengas activo.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `FTPClient.login (user, password)` realiza la conexión a un servidor FTP como el usuario indicado.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, lo que realiza es la validación del usuario, no la conexión al servidor.

4.4.- Programación de un cliente Telnet

Telnet (TELEcommunication NETwork) es un protocolo que permite acceder a otro equipo de la red y administrarlo de forma remota, esto es, como si estuviéramos sentados delante de él.



Q Ministerio de Educación. (Uso educativo-nc)

El protocolo Telenet está basado en:

- El modelo cliente/servidor, por lo que su esquema básico de funcionamiento será el típico de esta arquitectura.
- El servidor escucha las peticiones por el puerto 23.
- Su funcionamiento es en modo texto.

Este servicio puede resultar muy útil para administrar por ejemplo equipos sin pantalla o teclado, o bien servidores apilados en un rack o que no estén físicamente presentes.

En la actualidad, no se suele utilizar, debido a la poca seguridad que ofrece, ya que toda la información que se intercambia entre servidor y cliente, incluidos usuario y contraseña, se transmiten en texto plano. Esto es así, porque Telnet se creó pensando en la facilidad de uso y no en la seguridad. En su lugar, se utiliza un servicio de acceso y control remoto basado en el protocolo SSH.

A continuación podrás ver un ejemplo de **programación de un cliente Telnet el cual emplea otra biblioteca diferente a la que proporciona la API de Apache** (org.apache.commons.net), que ya descargaste en apartados anteriores:

Ejemplo Java de un cliente Telnet con librerías no pertenecientes al Apache Commons 

En caso de emplear las librerías de Apache Commons la biblioteca necesaria es org.apache.commons.net.telnet. Para disponer de ella tendremos que agregar a las dependencias del proyecto, como en el ejemplo de FTP, el archivo commons-net-n.x.x.jar. Entre las clases que proporciona para programar un cliente para el protocolo Telnet tenemos:

- **Clase [TelnetClient](#)**. Permite implementar un terminal virtual para el protocolo Telnet. Hereda de la clase [SocketClient](#)
 - El método [SocketClient.connect\(\)](#) que realiza la conexión al servidor
 - Los métodos [TelnetClient.getInputStream\(\)](#) y [TelnetClient.getOutputStream\(\)](#) que permiten, a través de objetos [InputStream\(\)](#) y [OutputStream\(\)](#), enviar y recibir datos a través de la conexión Telnet.
 - El método [TelnetClient.disconnect\(\)](#) que cierra la conexión al servidor así como los flujos de entrada y salida abiertos y el [socket](#) de comunicación.

En el siguiente enlace puedes ver el código de programación de un cliente Telnet. (Puedes utilizar las bibliotecas de Apache para Telnet o bien las indicadas en el ejemplo).

En el siguiente [ENLACE](#)  (5.8 KB) tienes un ejemplo de programación de un cliente Telnet (empleando las librerías de Apache Commons) el cuál se conecta a un servidor e interacciona con él. Ten en cuenta que para poder ejecutarlo puede que tengas que desactivar cualquier [cortafuegos](#)  o firewall que tengas activo.



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `telnet.connect("Miservidor")` realiza una conexión TelnetTelnet al servidor remoto Miservidor.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, falta especificar el puerto 23.



Para saber más

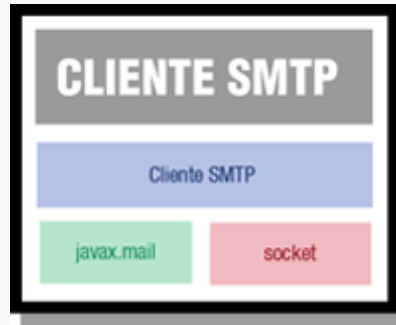
En el siguiente [HTML](#)  [ENLACE](#) dispones de toda la información sobre la biblioteca del API de Apache para el servicio Telnet.

Si quieres conocer más sobre el servicio SSH, consulta este [ENLACE](#) .

4.5.- Programación de un cliente SMTP

Vamos a ver dos ejemplos de programación de clientes SMTP. Uno de ellos basado en `sockets` y otro realizado mediante el API `javax.mail`.

En el primer caso, usando `sockets`, dispones de un ejemplo sencillo y resumido que puedes consultar en el siguiente enlace [ENLACE](#) (3.7 KB):



Q Ministerio de Educación. (Uso educativo nc)

En el segundo caso, utilizaremos el API `javax.mail`. Este proporciona las clases necesarias para implementar un sistema de correo. Vamos a destacar las **clases y métodos del `javax.mail` que nos permitirán crear nuestro cliente de correo:**

- **Clase `Session`**. Representa una sesión de correo. Agrupa las propiedades y valores por defecto que utiliza el API `javax.mail` para el correo.
 - Método `getDefaultInstance()`. Obtiene la sesión por defecto. Si no ha sido configurada, se creará una nueva de manera predeterminada. El parámetro que se le pasa debe recoger al menos las siguientes propiedades: protocolo y servidor **smtp**, puerto para el `socket` de sesión y tipo, usuario y puerto **smtp**).
- **Clase `Message`**. Modela un mensaje de correo electrónico.
 - Método `setFrom()`. Asigna el atributo `From` al mensaje, siendo éste la dirección del emisor.
 - Método `setRecipients()`. Asigna el tipo y direcciones de destinatarios.
 - Método `setSubject()`. Para indicar asunto del mensaje.
 - Método `setText()`. Asigna el texto o cuerpo del mensaje.
- **Clase `Transport`**. Representa el transporte de mensajes. Hereda de la clase `Service`, la cual proporciona funcionalidades comunes a todos los servicios de mensajería, tales como conexión, desconexión, transporte y almacenamiento.
 - Método `send()`. Realiza el envío del mensaje a todas las direcciones indicadas. Si alguna dirección de destino no es válida, se lanza una excepción `SendFailedException`.

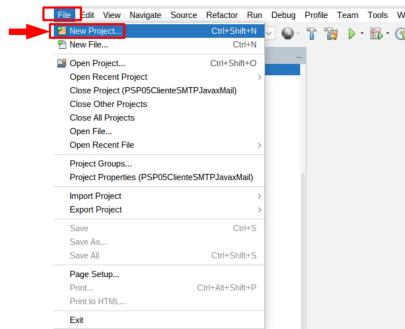
Agregar la dependencia de `javax.mail` en el proyecto de NetBeans es similar al proceso descrito en el apartado 4.2 pero esta vez el archivo `.jar` será descargado desde el apartado para JavaMail en el subdominio de JEE en GitHub. Es por ello que ahora, en la siguiente presentación, te indicamos los pasos a seguir para integrar la dependencia en NetBeans mediante una modificación del archivo `pom.xml` para lo cual el proyecto será de tipo `Java with Maven`:

API javax.mail

Integración en proyecto 'Java with Maven' en NetBeans 22

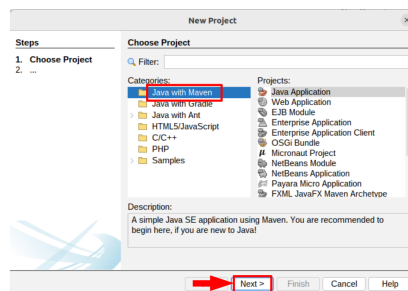
Creación de proyecto 'Java with Maven' (I)

- En el IDE NetBeans, en el menú 'File', hacemos clic en 'New Project...'



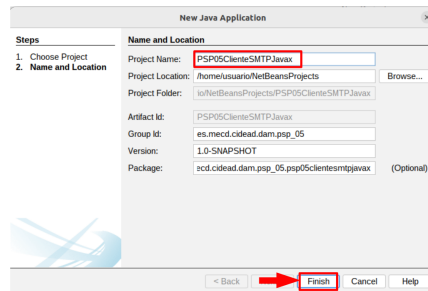
Creación de proyecto 'Java with Maven' (II)

- En el siguiente paso debemos indicar que deseamos crear un nuevo proyecto del tipo 'Java with Maven...' (es la opción marcada por defecto) y hacemos clic en el botón 'Next >' (siguiente):



Creación de proyecto 'Java with Maven' (III)

- En el siguiente paso debemos el nombre del proyecto en el apartado 'Java with Maven...' y, tras eso, hacemos clic en el botón 'Finish' (finalizar):



Configurar Maven en el proyecto (I)

- En el siguiente paso debemos configurar la nueva dependencia en Maven. Se trata de una sencilla, pero muy potente, herramienta para gestionar librerías y que se encarga de localizarla así como descargarla y dejarla lista en nuestro proyecto para que luego sea posible aplicarla en el código fuente del proyecto.
- Deberemos desplegar la pestaña 'Project Files' y, sobre el elemento etiquetado como 'pom.xml' hacer doble clic para que se abra la ventana de edición.



Configurar Maven en el proyecto (II)

- Ahora toca agregar, en el editor de código fuente, las sentencias necesarias para que el archivo 'pom.xml' pueda localizar, descargar e instalar la dependencia
- Deberemos copiar las siguientes sentencias (proporcionadas en la propia web de Javax.Mail:

```
<dependencies>
<dependency>
<groupId>com.sun.mail</groupId>
<artifactId>javax.mail</artifactId>
<version>1.6.2</version>
</dependency>
</dependencies>
```

Los valores que deben ser establecidos son los siguientes:

- clave 'groupId': valor 'com.sun.mail'
 - clave 'artifactId': valor 'javax.mail'
 - clave 'version': valor '1.6.2'
- (varía según aparecen nuevas versiones)

Configurar Maven en el proyecto (III)

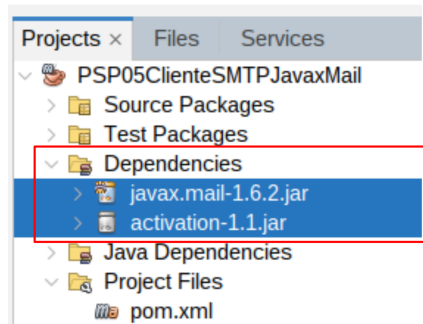
- Finalmente este es el aspecto que tiene el archivo 'pom.xml' al resto de las dependencias:





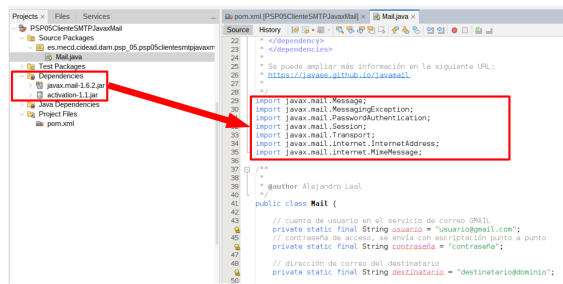
Configurar Maven en el proyecto (IV)

- Una vez guardemos los cambios podremos ver las nuevas librerías disponibles para emplear en el código fuente:



Importar en el código fuente los paquetes

- Ahora disponemos de los paquetes de la librería en los archivos de código fuente de cara a utilizarlos:



Credenciales	
Imagen	Datos licencia
Capturas de pantalla de todas las diapositivas de esta presentación	Autoría: Alejandro Leal Cruz Tipo de licencia y procedencia: capturas de pantalla del IDE NetBeans 22 (Apache License)

En el siguiente [ENLACE](#) dispones del proyecto java completo. Observa que no se realiza de forma explícita una conexión y desconexión, esto es debido a que va implícito en el objeto `Transport`. Cuando ejecutes el programa, recuerda poner los datos reales de tu cuenta de Gmail (o bien crea una) así como un destinatario válido (puede ser tu misma cuenta de Gmail). Observa también que el protocolo SMTP no utiliza el puerto 25, esto es porque se trata de SMTP seguro (SMTP sobre SSL).



Para saber más

Consulta el siguiente [ENLACE](#) para conocer a muchas otras posibilidades de programación de sistemas de correo con el API `javax.mail`.

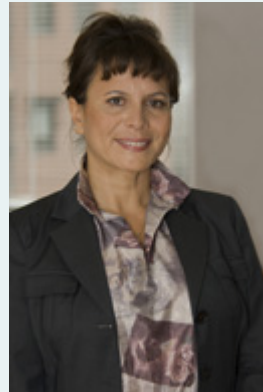
5.- Programación de servidores



Caso práctico

Hoy **Ada** ha se ha reunido con **Juan** y **Ana** para ver la marcha del proyecto. **Ana** le ha comentado que van a revisar el servidor web de la aplicación, pues ya saben que no gestiona bien la concurrencia de peticiones, y que además, tarda mucho en procesar algunas de ellas.

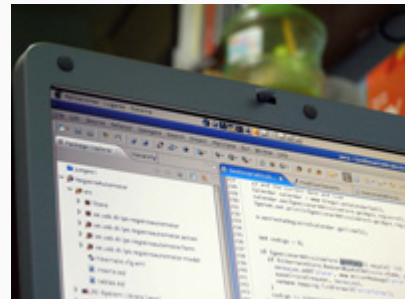
Ada les ha insistido en que además de solucionar el problema de los accesos simultáneos, deben conseguir que todos los servicios programados estén optimizados en cuanto a tiempos de respuesta. **Juan** y **Ana**, la miran y dicen -de acuerdo **Ada**, lo tendremos en cuenta.



Q Minist.de Educación (Uso educ.ncnc)

Entre los diferentes aspectos que se deben tener en cuenta cuando se diseña o se programa un servidor o servicio en red, vamos a resaltar los siguientes:

- El servidor debe poder **atender a multitud de peticiones que pueden ser concurrentes en el tiempo**. Esto lo podemos conseguir mediante la programación del servidor utilizando hilos o Threads.
- Es importante **optimizar el tiempo de respuesta del servidor**. Esto lo podemos controlar mediante la monitorización de los tiempos de proceso y transmisión del servidor.



Q RO ___Garcia (CC BY-SA)

La clase `ServerSocket` es la que se utiliza en Java a la hora de crear servidores. Para programar servidores o servicios basados en protocolos del nivel de aplicación, como por ejemplo el protocolo HTTP, será necesario conocer el comportamiento y funcionamiento del protocolo de aplicación en cuestión, y saber que tipo de mensajes intercambia con el cliente ante una solicitud o petición de datos.

En los siguientes apartados vamos a ver el ejemplo de cómo programar un servidor web básico, al que después le añadiremos la funcionalidad de que pueda atender de manera concurrente a varios usuarios, optimizando los recursos.



Para saber más

En el siguiente [ENLACE](#)  puedes ver el código completo de la programación del servidor y el cliente de un servicio básico de chat.



Citas para pensar

"Sin emoción no hay proyecto."

Eduardo Punset

5.1.- Programación de un servidor HTTP

Antes de lanzarnos a la programación del servidor HTTP, vamos a recordar o conocer cómo funciona este protocolo y a sentar las hipótesis de trabajo.

El servidor que vamos a programar cumple lo siguiente:

- Se basa en la versión 1.1 del protocolo HTTP.
- Implementará solo una parte del protocolo.
- Se basa en dos tipos de mensajes: peticiones de clientes a servidores y respuestas de servidores a clientes.
- Nuestro servidor solo implementará peticiones `GET`.



Q MjZ Photography (CC BY-SA)



Debes conocer

¿Cómo procesa el servidor web las peticiones del cliente? Dependerá del método utilizado en la petición por el cliente, que en nuestro ejemplo es el `GET`. Consulta la siguiente web para ver cómo se hace.

[HTML Procesamiento de peticiones HTTP](#)

Para crear un servidor web que implemente el protocolo HTTP deberá seguir el siguiente comportamiento básico:

- Crear un `socketServer` asociado al puerto 80 (puerto por defecto para el protocolo HTTP).
- Esperar peticiones del cliente.
- Aceptar la petición del cliente.
- Procesar petición (intercambio de mensajes según protocolo + transmisión de datos).
- Cerrar `socket` del cliente.

A continuación vamos a programar un sencillo servidor web HTTP que acepte peticiones por el puerto 8500 de un cliente que será tu propio navegador web. Según la URL que solicites desde el navegador el servidor contestará con diferente información. Los casos que vamos a contemplar son los siguientes:

- Al solicitar desde tu navegador `http://localhost:8500` te dará la bienvenida.
- Al solicitar desde tu navegador `http://localhost:8500/loremipsum` mostrará un párrafo de texto de relleno del tipo *Lorem Ipsum*.

- Al solicitar desde tu navegador un recurso diferente a los anteriores, como por ejemplo `http://localhost:8500/a`, mostrara un mensaje de error.

Es importante que, una vez hayas probado el servidor, detengas su ejecución antes de volver a iniciarlo.

En el siguiente [ENLACE](#) dispones del proyecto Java desarrollado en NetBeans:



Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `GET` lo utiliza el servidor para enviar información al cliente.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, es un método que utiliza el cliente para realizar la petición al servidor.



Para saber más

En el siguiente [ENLACE](#) dispones de la información necesaria para conocer el funcionamiento del protocolo HTTP.

5.2.- Implementar comunicaciones simultáneas

De cara a que el servidor anteriormente implementado pueda proporcionar la funcionalidad de atender comunicaciones y solicitudes simultáneas es necesario utilizar hilos o threads.

En un contexto realista de explotación cualquier servidor HTTP tiene que ser capaz de atender varias peticiones simultáneamente. Es por ello que tenemos que ser capaces de modificar su código de forma que pueda utilizar varios hilos de ejecución de forma paralela. Es posible conseguirlo de la siguiente manera:



Q Luis Perez (CC BY-SA)

- El hilo principal (o sea, el que inicia la aplicación) creará el `socket` servidor que permanecerá a la espera de que llegue alguna petición (estado de escucha permanente).
- Cualquier solicitud recibida por parte de algún cliente es aceptada y el hilo principal le asignará un `socket` cliente para comunicarse y poder enviarle la respuesta. Dar respuesta a esta necesidad implica que, en lugar de atender dicha solicitud él mismo, el hilo principal deberá crear un nuevo hilo hijo que será quien realice la tarea de despacho empleando para ello el `socket` cliente que le fue asignado desde el hilo principal. Esta es la manera de que el hilo principal pueda estar siempre disponible a la espera de nuevas peticiones de nuevos clientes que realicen nuevas solicitudes.

De forma resumida el **código del hilo principal** tendrá el siguiente aspecto:

```
try {
    socketServidor = new ServerSocket(puerto);

    while (true) {
        // acepta la solicitud del cliente y asigna un socket client
        // para enviar la respuesta al cliente
        socketCliente = socketServidor.accept();

        // crea un nuevo hilo donde realiza las operaciones necesari
        // despachar la solicitud y le pasa el socketCliente
        hilo = new HiloCliente(socketCliente);

        hilo.start(); // inicio de la ejecución del hilo cliente
    }
} catch (IOException ex) {
    System.out.println("Error: " + ex.getMessage());
}
```

En el extracto anterior la clase `HiloCliente` será una clase Java que extienda a la clase `Thread` y cuyo constructor deberá almacenar el `socket` `socketCliente` recibido desde el hilo principal en una variable local. Ésta será

empleada más adelante por el método `run()` con el objetivo de enviar la respuesta al cliente:

```
public class HiloCliente extends Thread {
    private socketCliente;

    public HiloCliente(Socket socketCliente) {
        this.socketCliente = socketCliente;
    }

    public void run() {
        try {
            // aquí se ubica la lógica que permite
            // construir la respuesta para la solicitud
            // del cliente

            // Tras eso emplea 'socketCliente' para enviar
            // la respuesta
        } catch (IOException ex) {
            System.out.println("Error: " + ex.getMessage());
        }
    }
}
```

Con el modelo de ejecución planteado anteriormente habremos logrado desarrollar un servidor HTTP realista, será capaz de gestionar peticiones de forma concurrente y con eficiencia.



Para saber más

Aquí tienes un [EJEMPLO](#) mucho más avanzado de un servidor HTTP completo que usa un pool de hilos para gestionar la concurrencia:

En este otro [ENLACE](#) puedes consultar la clase `Thread` de Java:

5.3.- Monitorización de tiempos de respuesta

Una cuestión muy importante para evaluar el rendimiento y comportamiento de una aplicación cliente/servidor es monitorizar los tiempos de respuesta del servidor. Los tiempos que intervienen desde que un cliente realiza una petición al servidor hasta que recibe su resultado son dos:

- **Tiempo de procesamiento.** Es el tiempo que el servidor necesita para procesar la petición del cliente y enviar los datos.
- **Tiempo de transmisión.** Es el tiempo que tardan los mensajes en llegar a su destino a través de la red.

```
// registro de la lectura del reloj del sistema antes de iniciar
// el procesamiento
long tiempoInicio = System.currentTimeMillis();

/**
 * procesamiento de la petición del cliente
 * ...
 * ...
 * ...
 */

// registro de la lectura del reloj del sistema después de finalizar el
// procesamiento
long tiempoFin = System.currentTimeMillis();
long tiempoProceso = tiempoFin - tiempoInicio;
System.out.println("Tiempo requerido en procesar la petición: " + tiempoProceso);
```

Q Ministerio de Educación (Uso educativo nc)

¿Cómo podemos medir esos tiempos?

Para **medir el tiempo de procesamiento** basta medir el tiempo que transcurre en el servidor para procesar la solicitud del cliente. Por ejemplo, el [código Java que mide el tiempo de procesamiento](#) permite medir ese tiempo en milisegundos.

Para **medir el tiempo de transmisión** será necesario que el servidor envíe un mensaje con el tiempo del sistema al cliente. El cliente, al recibir el mensaje, debe calcular su tiempo de sistema y compararlo con el tiempo recibido en el mensaje.

Para poder comparar los tiempos de respuesta de dos equipos es imprescindible que sus relojes de sistema estén sincronizados a través de cualquier servicio de tiempo (NTP). En equipos Windows, en cambio, dicha tarea de sincronización de relojes se realiza de forma automática.



Autoevaluación

Señala la opción correcta. Para monitorizar el tiempo de respuesta del servidor:

- ☐ Basta con controlar el tiempo de proceso del servidor.
- ☐ Solo interesa realizar esta tarea si hay sobrecarga.
- ☐ Solo depende del tiempo de transmisión.

- ☐ Se debe tener en cuenta el tiempo de proceso del servidor y el tiempo de transmisión.

No es correcto, prueba de nuevo.

Incorrecto, deberías haber leído mejor.

No es cierto, debes prestar más atención.

Muy bien, vamos por buen camino.

Solución

1. Incorrecto (#answer-80_63)
2. Incorrecto (#answer-80_103)
3. Incorrecto (#answer-80_106)
4. Opción correcta (#answer-80_109)



Para saber más

Consulta el siguiente [ENLACE](#) para obtener más información sobre el servicio NTP.

5.4.- Ejemplo de monitorización del tiempo de transmisión

Vamos a ver un ejemplo de cómo calcular el tiempo de transmisión de un recurso desde un servidor web. Para ello, vamos a modificar el cliente HTTP realizado anteriormente con las clases `URL` y `URLConnection`, para añadir la función `URLConnection.getDate()` que proporciona el instante en el que el servidor inicia la transmisión de su respuesta al cliente. Ese tiempo es enviado por el servidor al cliente mediante la cabecera `Date`, al igual que la cabecera que vimos anteriormente sobre el tipo de contenido, y que se podía recuperar con el método `URLConnection.getContentType()`.



Q Fco.José Gómez (CC BY-SA)

Desde el siguiente [ENLACE](#) puedes descargar el proyecto Java completo.



Citas para pensar

"Lo que tenemos que aprender, lo aprendemos haciendo."

Aristóteles



Para saber más

Mediante los comandos `ping` y `tracer` puedes medir el tiempo que tarda un servidor en responder a tu petición. Consulta el siguiente [ENLACE](#) para más información.